

I have added instructions at the top of the prompt telling the AI agent to execute this in stages so it doesn't get overwhelmed and break your code.

Copy and paste everything in the block below into your AI editor:

Project Goal: Build a local Windows desktop application (.exe) for a Lithium-ion battery trading business in Nepal. The app manages imports from India, B2B/B2C sales, a double-entry accounting ledger (like Tally), bank loans, and predictive data analytics.

Tech Stack:

- **Backend/AI:** Python, FastAPI, SQLite (local database), SQLAlchemy, Pandas, Scikit-learn, PyInstaller.
- **Frontend:** Next.js (TypeScript, Tailwind CSS, Recharts) configured for Static HTML Export.
- **Desktop Wrapper:** Electron & electron-builder.

Agent Instructions: Do not attempt to write all code at once. Execute this step-by-step. Complete Step 1, verify the files are created, and then proceed to the next step.

Step 1: Project Setup & SQLite Database Schema

1. Initialize a Python backend directory (backend/) and a Next.js frontend directory (frontend/).
2. Create the SQLite database setup using SQLAlchemy in backend/app/database.py pointing to a local database.db file.
3. Write SQLAlchemy models in backend/app/models.py for:
 - Customer: id, name, type (Individual, Dealer, Rickshaw, Solar), phone.
 - Inventory: id, sku, model, cost_price, selling_price, stock_quantity.
 - AccountHead: id, code, name, category (Asset, Liability, Equity, Revenue, Expense).
 - JournalEntry & JournalLine: Standard double-entry ledger format.
 - BankLoan: id, principal_amount, disbursed_date, interest_rate (default 10%), account_id.
 - LoanRepayment: id, loan_id, payment_date, principal_paid, interest_paid.

Step 2: Seed Data Generator

Write a script backend/app/seed.py that populates database.db with realistic dummy data for the last 12 months: 5 bank loans, 10 battery SKUs, 30 customers, and 200+ balanced journal entries (incorporating seasonal spikes and loan payments).

Step 3: Tally-Style Data Entry & Core APIs

1. **Backend:** Create CRUD endpoints for Inventory, Customers, and Journal Entries. Ensure the Journal endpoint validates that total debits equal total credits. All currency must format as NPR.
2. **Frontend:** Build a high-density, keyboard-navigable 'Daily Journal' data grid in Next.js. Users should be able to tab through Date, Account Head (dropdown), Particulars, Debit, and Credit rapidly.

Step 4: Bank Loan & 10% Interest Tracking

1. **Backend:** Create GET /api/loans/summary to calculate days elapsed since disbursement and accrue simple interest at a strict 10% annual rate. Calculate Total Outstanding Balance.
2. **Frontend:** Build a 'Bank Loans' UI showing Borrowed, 10% Interest Accrued, Repaid, and Outstanding amounts. Include a modal to record repayments which auto-posts to the ledger.

Step 5: Data Analysis & Forecasting

1. **Backend Analytics:** Create GET /api/analytics that accepts a timeframe query (1_week, 1_month, 3_months, 6_months, 1_year) and aggregates revenue, expenses, and profits.
2. **Backend ML:** Create GET /api/forecast. Use Pandas and Scikit-learn (Linear Regression or Exponential Smoothing) on the SQLite historical data to predict battery sales and revenue for the next 1_week through 1_year.
3. **Frontend:** Build an Analytics Dashboard using Recharts. Include a line chart showing Historical Data (solid line) connecting to Predicted Future Data (dashed line), and a bar chart showing sales broken down by customer type. Add a toggle for the 1W/1M/3M/6M/1Y timeframes.

Step 6: Next.js Static Export & PyInstaller

1. Modify frontend/next.config.js to enable output: 'export'.
2. Ensure all frontend API calls point to http://localhost:8000.

3. Create a build script `build_backend.py` that uses `PyInstaller` to compile the FastAPI app into a standalone `.exe`. Ensure it bundles necessary libraries like `pandas` and `scikit-learn`.

Step 7: Electron Desktop Wrapper

1. Initialize an Electron project in a new folder `desktop-app/`.
2. Write `main.js` to:
 - Spawn the `PyInstaller backend.exe` as a hidden background process on port 8000.
 - Wait for port 8000 to be active.
 - Open the main Electron window and load the Next.js static `index.html`.
 - Kill the backend process gracefully when the Electron window is closed.
3. Configure `electron-builder` in `package.json` to bundle the static frontend, the Electron runner, and the compiled `backend.exe` into a single `Setup.exe` installer for Windows.